Apuntes del Sprint 8 · Duelo de modelos: árboles, ensembles, SVM y cómo elegir campeón

ANTES DE EMPEZAR

este sprint usa lo de los Sprints 6 y 7 (pipelines, validación cruzada, AUC, precisión y recall, sobreajuste). Si algo te suena raro, repasa esos apuntes.

Las diapositivas del Sprint 8 presentan **cuatro familias de modelos y una competición** en muy poco espacio. Estos apuntes **explican la idea de cada una con ejemplos pequeños que puedes calcular a mano**. Léelos antes de cada cuaderno.

0. De qué va el sprint (en 1 minuto)

TurisData ya sabe que **se pueden predecir las cancelaciones**. Ahora la pregunta es: **¿qué modelo ponemos en producción?** Hay tres criterios, y rara vez gana el mismo en los tres:

Criterio	Pregunta	Cómo se mide
Rendimiento	¿El más preciso?	AUC y *average precision* en validación cruzada
Explicabilidad	¿Lo puede entender el gerente?	Puntuación 1-5 que pones tú
Coste	¿El más barato de mantener?	Tiempo de ajuste y de predicción, tamaño del modelo

Un **leaderboard** (tabla de clasificación) mide a todos los modelos **con la misma vara**.

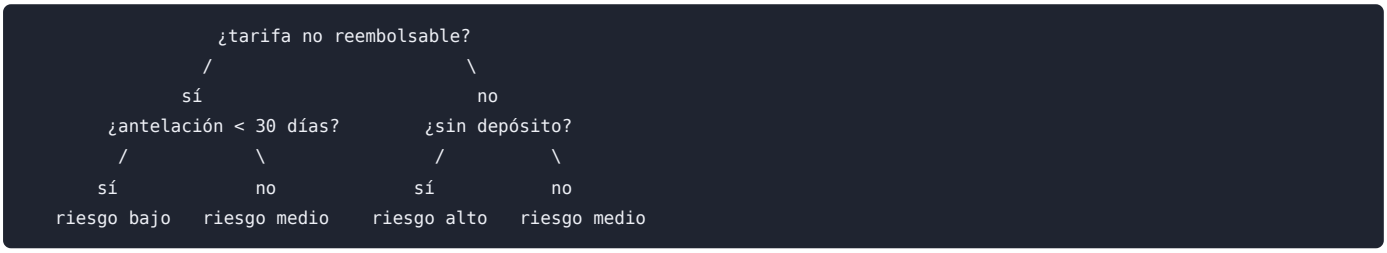
Presentación	Familia	Cuaderno
S08.1	Árbol de decisión	S08_01
S08.2	Random Forest y boosting	S08_02, S08_03
S08.3	SVM, búsqueda de hiperparámetros y cierre	S08_04, S08_05, S08_06

El reto: el **duelo de modelos**: competís con varios, elegís un **campeón justificado** y lo documentáis en una **model card**.

DÍA 1 · Árboles de decisión

1. La idea: preguntas de sí/no encadenadas

Un **árbol de decisión** es como el juego de las **veinte preguntas**: va haciendo preguntas de sí/no hasta poder decidir.



- Cada **nodo** es una pregunta; cada **hoja** (final de rama) es la predicción.
- **No necesita escalar** las variables y **cada predicción se explica con un camino de preguntas**: eso es su gran ventaja.

¿Cómo elige la mejor pregunta? El índice de Gini

El árbol busca, entre todas las variables y umbrales, **la pregunta que mejor separa** las canceladas de las no canceladas. Mide la «mezcla» de un grupo con el **índice de Gini**: **0 = grupo puro** (todos de la misma clase); **0,5 = máxima mezcla** (mitad y mitad).

Ejemplo con 20 reservas (10 canceladas, 10 no): Gini inicial = $1 - (0{,}5^2 + 0{,}5^2)$ = **0,5**.

Pregunta: *¿la tarifa es no reembolsable?*

Rama	Reservas	Canceladas	No canc.	Gini
Sí	8	1	7	$1 - (0{,}125^2 + 0{,}875^2)$ = **0,219**
No	12	9	3	$1 - (0{,}75^2 + 0{,}25^2)$ = **0,375**

Gini ponderado después = (8/20)·0,219 + (12/20)·0,375 = 0,0875 + 0,225 = **0,3125**.

Mejora = 0,5 − 0,3125 = 0,1875. Cuanta más mejora, mejor pregunta. El árbol **prueba todas las preguntas posibles y elige la de mayor mejora**, y repite en cada rama.

```
variable_raiz = nombres[arbol_peq.tree_.feature[0]]     # la variable de la primera pregunta
n_hojas = arbol_peq.get_n_leaves()
```

2. Sobreajuste: el árbol libre

Sin límites, el árbol **sigue preguntando hasta que cada hoja es pura**. Eso significa **memorizar** el entrenamiento, incluido el ruido (por ejemplo, una hoja que existe solo para 2 reservas raras).

	AUC en entrenamiento	AUC en prueba
Árbol libre	**1,00**	0,68 *(ilustrativo)*

La señal de alarma, como siempre: una gran brecha entre entrenamiento y prueba (Sprint 1).

3. Poda: frenar el crecimiento

Hiperparámetro	Qué limita
`max_depth`	N° de preguntas seguidas
`min_samples_leaf`	**Mínimo de reservas por hoja** (evita hojas «de 2 casos»)
`ccp_alpha`	**Poda a posteriori**: recorta ramas que no compensan

Se elige el valor con **validación cruzada**: `mejor_min_hoja = hojas_min[np.argmax(auc_cv_hojas)]`.

4. Importancia de variables... y su gran defecto

```
importancias = pd.Series(arbol_podado.feature_importances_, index=nombres).sort_values(ascending=False)
```

- La importancia **suma 1** y dice **qué mira el modelo**, **no qué causa** el resultado.
- **Tiende a inflar las variables numéricas** con muchos valores distintos (tienen más sitios donde «cortar»).
- Es **inestable**: con submuestras del 80 % salen 5 árboles que **predicen distinto las mismas reservas**. Eso es lo que arreglan los *ensembles*.

Mini-reto: reglas para el gerente

```
arbol_gerente = DecisionTreeClassifier(max_depth=3, min_samples_leaf=50, random_state=SEMILLA).fit(X_train, y_train)
print(export_text(arbol_gerente, feature_names=nombres))
```

`export_text` imprime el árbol como reglas. Tu trabajo es **traducir a lenguaje de negocio**: *«Si la tarifa es no reembolsable y la antelación es corta, el riesgo de cancelación es bajo»*.

> **Pregunta para pensar:** si el gerente pregunta «¿por qué se marcó esta reserva?», ¿qué ventaja tiene un árbol frente a un modelo de 300 árboles? *(Respuesta: se le puede enseñar el camino de 3-4 preguntas.)*

DÍA 2 · Ensembles: juntar muchos árboles

Un árbol solo es inestable. Muchos árboles juntos son más precisos y estables, a cambio de **explicabilidad y coste**. Es la «sabiduría de la multitud»: la media de muchos expertos imperfectos suele acertar más que uno solo.

5. Dos estrategias

	Bagging → Random Forest	Boosting
Idea	Muchos árboles **independientes** que se **promedian**	Árboles pequeños **en cadena**, cada uno **corrige los errores del anterior**
Cómo	Cada árbol ve una muestra con reemplazo y **variables al azar**	Cada árbol se suma con un paso pequeño (`learning_rate`)
Analogía	**Votación** de un jurado	**Tutor** que repasa lo que fallaste

6. Random Forest

Si todos los árboles fueran iguales, promediar no serviría de nada. Por eso se introduce **aleatoriedad**: cada árbol ve **una muestra distinta** de reservas y **solo algunas variables** en cada pregunta.

Ejemplo: tres árboles dan para una reserva 0,90; 0,40 y 0,80 de probabilidad de cancelar. El bosque promedia: **(0,9 + 0,4 + 0,8) / 3 = 0,70**. El árbol que se equivocó (0,40) queda diluido por los otros.

```
bosque = RandomForestClassifier(n_estimators=300, min_samples_leaf=5, random_state=SEMILLA, n_jobs=2).fit(Xtr, y_train)
```

Hiperparámetro	Qué hace
`n_estimators`	Nº de árboles. **Más árboles casi nunca perjudica**; se estabiliza (de 100 a 400 la mejora es mínima)
`min_samples_leaf` , `max_depth`	Controlan el sobreajuste **de cada árbol**
`max_features`	Cuántas variables se miran en cada pregunta

Puntuación OOB (*out-of-bag*): cada árbol **no vio** ~⅓ de las reservas. Con esas reservas «de repuesto» se obtiene **una validación gratis**, sin separar nada.

Importancia: impureza frente a permutación

	Impureza (`feature_importances_`)	Permutación
Cómo	Lo que **reduce el Gini** en el entrenamiento	Se **baraja una columna del test** y se mide **cuánto cae el AUC**
Pega	**Sesgada** a numéricas con muchos valores	Más lenta
Ventaja	Rápida	**Mide el efecto real** sobre la predicción

Idea de la permutación: si barajas la columna `tarifa` (rompiendo su relación con el resultado) y el AUC cae de 0,80 a 0,70, esa columna **importaba**. Si barajas y no cambia nada, **no hacía falta**.

> **El bosque gana rendimiento y pierde explicabilidad:** 300 árboles no se leen. Además cuesta más **tiempo y memoria**.

7. Boosting: corregir errores en cadena

1. Un **primer árbol muy pequeño** predice.
2. El siguiente se entrena **para predecir los errores** (residuos) del anterior.
3. Se **suma multiplicado por** `learning_rate`. Se repite cientos de veces.

Ejemplo de cuentas: precios reales `[100, 120, 140]`. Primera predicción: la media, 120 para todos. Con `learning_rate = 0,5`:

Ronda	Predicciones	Error medio (MAE)
0 (la media)	120, 120, 120	13,3
1	110, 120, 130	6,7
2	105, 120, 135	3,3

En cada ronda el árbol aprende los **residuos** (−20, 0, +20) y **corrige la mitad** (por el `learning_rate`). **El error se reduce a la mitad cada vez.**

El peligro de pasarse

Con demasiados árboles, el AUC de validación **sube y luego baja**: **sobreajuste**. La solución es la **parada temprana** (*early stopping*): fijas un máximo alto de árboles, y el entrenamiento **se detiene si la validación no mejora en N rondas**. Se usa una **tasa baja** (0,03-0,1) y más árboles.

```
mejor_n_arboles = int(np.argmax(auc_val_por_arbol)) + 1
caida_final = max(auc_val_por_arbol) - auc_val_por_arbol[-1]     # cuánto se pierde por pasarse
```

XGBoost y LightGBM

Dos implementaciones muy rápidas y populares del boosting:

	XGBoost	LightGBM
Crece	**Por niveles** (`max_depth`)	**Por hojas** (`num_leaves`)
Control de hojas	`min_child_weight`	`min_child_samples`
Parada temprana	`early_stopping_rounds` + `eval_set`	`callbacks` + `eval_set`
Importancias	Por ganancia	Por ganancia

Se usa un **`eval_set`** (un conjunto de validación) para que el modelo sepa cuándo parar.

8. El leaderboard: primeros modelos

```
registrar(evaluar_modelo("baseline", "referencia", hacer_pipe(DummyClassifier(strategy="prior")), 5, notas))
```

`evaluar_modelo` mide: **AUC y AP** (validación cruzada), **tiempo de ajuste**, **tiempo de predicción por 1.000 reservas** y **tamaño en KB**. **La explicabilidad (1-5) la pones tú.**

Primero se registran: baseline, logística, árbol y bosque. SVM y búsqueda, después.

DÍA 3 · SVM, hiperparámetros y elegir campeón

9. SVM: la idea en dos dimensiones

Imagina cada reserva como **un punto en un plano** (por ejemplo, `antelacion_dias` y `precio_noche`). Una **SVM** (máquina de vectores de soporte) traza **la recta que separa las dos clases dejando el «pasillo» (margen) más ancho posible**.

Ejemplo en una línea: las no canceladas están en x = 1 y 2; las canceladas en x = 5 y 6. La frontera se coloca **en el punto medio entre el 2 y el 5**, en **3,5**; el pasillo mide 3.

- Los puntos que **tocan el borde del pasillo** (el 2 y el 5) son los **vectores de soporte**.
- **Solo ellos definen la frontera**: si quitaras el 1 o el 6, la frontera no se movería.

Dos botones: `C` y el kernel

Botón	Qué controla
C	Cuánto se castiga un punto mal clasificado. **C grande → margen estrecho, ajusta mucho** (riesgo de sobreajuste). **C pequeño → margen ancho, más tolerancia a errores**
Kernel	¿Y si una recta no basta? `"linear"` → frontera recta. `"rbf"` → «curva» el espacio y dibuja **fronteras curvas**
gamma (solo RBF)	Cuánto influye cada punto. **Alto → «islotes»** alrededor de cada punto (sobreajuste). **Bajo → casi una recta** (infraajuste)

Ejercicio de «las dos lunas»: dos grupos con forma de luna que una recta **no puede** separar. El kernel RBF **supera al lineal** porque curva la frontera. Con `gamma = 0,05 / 1 / 50` verás **infraajuste → ajuste razonable → sobreajuste**.

Con datos reales: hay que escalar

Una SVM **mide distancias**: si `precio_noche` va de 40 a 400 y `peticiones_especiales` de 0 a 3, la primera lo domina todo. **Escalar mejora claramente el AUC.**

```
Pipeline([("prep", prep_escalado()), ("svm", SVC(kernel="rbf", C=1))])
```

Cuándo usar (y cuándo no) una SVM

Sí	No
Pocas filas y muchas variables	**Muchas filas** (millones): el tiempo crece más que linealmente
Fronteras curvas que una recta no capta	Hay que **explicar cada decisión** al cliente
Variables ya escaladas y limpias	Necesitas **probabilidades rápidas** (`probability=True` entrena ~5 veces más despacio)

En el duelo de TurisData, **la SVM lineal rinde casi igual que la RBF y es más sencilla**.

10. Parámetros e hiperparámetros

	Parámetros	Hiperparámetros
Quién los fija	**El modelo**, al entrenar	**Nosotros**, antes de entrenar
Ejemplos	Coeficientes, umbrales de un árbol	`max_depth`, `learning_rate`, `C`, `k`, `alpha`

Analogía: el modelo es un **horno**. Los **hiperparámetros** son los botones (temperatura, tiempo) que tú pones. Los **parámetros** son lo que ocurre dentro (cómo se hace el pan).

11. Buscar hiperparámetros: rejilla frente a azar

```
rejilla = {"rf__n_estimators": [100, 200], "rf__max_depth": [4, 8], "rf__min_samples_leaf": [10, 40]}
busqueda_grid = GridSearchCV(pipe_rf, rejilla, cv=5, scoring="roc_auc")
```

El prefijo `rf__` indica **a qué paso del Pipeline** pertenece el hiperparámetro.

	GridSearchCV (rejilla)	RandomizedSearchCV (aleatoria)
Qué prueba	**Todas las combinaciones**	**n combinaciones al azar**
Ejemplo	2 × 2 × 2 = **8** combinaciones × 5 folds = **40 entrenamientos**	25 combinaciones × 5 folds = **125 entrenamientos**
Ventaja	Exhaustiva	**Explora más valores distintos** con el mismo presupuesto

```
distribuciones = {"lgb__learning_rate": loguniform(0.01, 0.3),
                  "lgb__num_leaves": randint(3, 40),
                  "lgb__min_child_samples": randint(10, 200)}
busqueda_rand = RandomizedSearchCV(pipe_lgb, distribuciones, n_iter=25, cv=5, scoring="roc_auc")
```

Leer los resultados sin engañarte

- En `cv_results_` **suele haber muchas configuraciones empatadas**. **Regla del 1 error estándar:** entre las empatadas, **elige la más sencilla**.
- **La nota de la búsqueda es optimista** (la «maldición del ganador»): has elegido la mejor *entre muchas*, así que parte de su nota es **suerte**. **El resultado honesto es el del test, usado una sola vez.**

12. Cierre del duelo: campeón, test y model card

1. **Registra** SVM y los modelos que falten en el leaderboard.
2. **Define tus pesos** de rendimiento, explicabilidad y coste (suman 1) y calcula la **puntuación ponderada**.
3. **Reconstruye el campeón** y evalúalo en **test, una sola vez**.
4. Rellena la **model card**: para qué sirve, con qué datos, cómo rinde y **qué no debe usarse**.

Ejemplo de puntuación ponderada

Imagina tres modelos con puntuaciones normalizadas de 0 a 1 (1 = mejor):

Modelo	Rendimiento	Explicabilidad	Coste (1 = barato)
Random Forest	0,95	0,20	0,40

Modelo	Rendimiento	Explicabilidad	Coste (1 = barato)
Árbol podado	0,70	1,00	0,90
Logística	0,75	0,90	1,00

Pesos (rend. / expl. / coste)	RF	Árbol	Logística	Campeón
0,5 / 0,3 / 0,2	0,615	0,830	**0,845**	Logística
0,8 / 0,1 / 0,1	**0,820**	0,750	0,790	**Random Forest**

Ejemplo del cálculo (Random Forest, primera fila): 0,5·0,95 + 0,3·0,20 + 0,2·0,40 = 0,475 + 0,06 + 0,08 = **0,615**.

> **¿Cambia el campeón si varían los pesos?** Sí, como ves. **Esa es la pregunta que el gerente querrá ver respondida**: que sepas decir *«con estas prioridades gana X; si las prioridades cambian, gana Y»*.

La *model card*: la «etiqueta nutricional» del modelo

Un documento corto que dice **para qué sirve**, **con qué datos se entrenó**, **cómo rinde** y **qué usos están desaconsejados**. Es parte de la **IA responsable** (Sprint 1): quien use el modelo debe saber **hasta dónde puede fiarse**.

---

13. Dónde practicar cada cosa

Si no entiendes...	Mira la sección	Practica en
Árbol, Gini, poda, importancia, reglas	1-4	`S08_01_arboles_decision`
Bosque, OOB, importancia por permutación	5-6	`S08_02_random_forest`
Boosting, parada temprana, XGBoost, LightGBM	7	`S08_03_boosting_xgboost_lightgbm`
SVM, `C`, kernel, `gamma`	9	`S08_04_svm`
Hiperparámetros, rejilla y azar	10-11	`S08_05_busqueda_hiperparametros`
Leaderboard, campeón, model card	8, 12	`S08_06_leaderboard_plantilla` y reto

14. Chuleta de vocabulario

Palabra	En lengua normal
Árbol de decisión	Preguntas de sí/no encadenadas hasta decidir
Gini	Medida de «mezcla» de un grupo (0 = puro)
Hoja	Final de una rama: la predicción
Poda	Limitar el árbol para que no memorice
Ensemble	Juntar muchos modelos
Bagging / Random Forest	Muchos árboles independientes que se promedian
Boosting	Árboles en cadena, cada uno corrige al anterior
OOB	Reservas que un árbol no vio: validación gratis
Importancia por permutación	Barajar una columna y ver cuánto cae el AUC
learning_rate	Tamaño del paso en el boosting

Palabra	En lengua normal
Parada temprana	Detener el entrenamiento cuando la validación deja de mejorar
SVM	Recta (o curva) que separa dejando el pasillo más ancho
Vectores de soporte	Los puntos del borde del pasillo
Kernel	Truco para curvar la frontera
Hiperparámetro	Botón que fijas tú antes de entrenar
GridSearchCV	Probar todas las combinaciones
RandomizedSearchCV	Probar combinaciones al azar
Leaderboard	Tabla de clasificación de modelos
Model card	Ficha que describe un modelo y sus límites

15. Cómo estudiar estos apuntes

1. **Antes de cada cuaderno**, lee la sección que indica la tabla del apartado 13.
2. **Haz a mano el ejemplo del Gini** (sección 1) y el de la **puntuación ponderada** (sección 12): son los que más ayudan a entender qué hace el ordenador.
3. **Para cada familia**, ten claro: *¿cómo decide? ¿cuándo se sobreajusta? ¿cuánto cuesta? ¿se puede explicar?*
4. **En el reto, no gana «el que más puntúa»:** gana el que **mejor se justifica** con rendimiento, explicabilidad y coste. Y siempre, el test **una sola vez**.